it easy to create a pipeline of these *middleware* apps such that they can be integrated into any Ruby web framework.

Here's a minimal example of Rack in action (rack-tion?):

web/rack_01/config.ru

```
require "bundler/inline"

gemfile do
  source "https://rubygems.org"
  gem "rack"
  gem "rackup"
end

run do |env|
  [200, {"content-type" => "text/plain"}, ["Welcome to Rack"]]
end
```

This is a configuration file for Rack, often called a "rackup file." This particular rackup file is doing two things.

First, it's using Bundler's inline function (see Single File Projects, on page 261) to ensure that the gems Rack and Rackup are installed. The Rack gem manages most of the Rack functionality; Rackup is the default command-line application for serving applications defined using Rack.

Second, it's creating a Rack app and running it using the Rack command `run`.

To make this work, in the directory with the code run this:

```
$ bundle install
$ rackup
[2022-09-25 10:04:10] INFO  WEBrick 1.7.0
[2022-09-25 10:04:10] INFO  ruby 3.1.2 (2022-04-12) [arm64-darwin21]
[2022-09-25 10:04:10] INFO  WEBrick::HTTPServer#start: pid=31252 port=9292
```

This has started a web server called WEBrick and it is serving at port 9292, which you can confirm by going into a browser and hitting `http://localhost:9292`. You will receive "Welcome to Rack" as the response.

**Rack Versions**



If you are familiar with Rack, this example may look a little strange. This example uses Rack 3.0, which is relatively new as this is being written. The rackup gem was extracted from Rack in 3.0 in part because WEBrick is now a separate project and is no longer part of the standard Ruby distribution. Also, the `run` method taking a block argument is new in 3.0.

That shows that it works, but what is Rack doing?

The `run` method is part of Rack. Typically, a Rack application defined in a rackup file will have one `run` method (though, as we'll see, you can specify multiple run methods if each one is matched to a different URL path). The `run` method takes as its argument either a block or any Ruby object that can respond to the `call` method. Because lambdas and Procs both define `call`, either of those can be arguments to `run`.

Here's what our app would look like with a stabby lambda: